Switch-Level Modeling in Verilog

1 Introduction

Switch-level modeling describes a digital circuit in terms of MOS transistors and switch primitives rather than only Boolean operators or logic gates. A transistor primitive behaves like a controlled connection: it conducts when enabled and otherwise leaves its output disconnected (high impedance, `Z`). Verilog provides ideal and resistive switch families; these examples use ideal primitives. This level of detail is useful for illustrating how transistor networks implement familiar logic functions, though it is less common for describing large design modules.

The examples use Verilog's built-in switch primitives and a few helper modules. The diagrams and handwritten notes from the source PDF are retained as figures; code that was embedded in screenshots is set as editable, copyable Verilog below.

2 Switch Primitives

2.1 NMOS and PMOS switches

An NMOS switch conducts when its control is 1, while a PMOS switch conducts when its control is 0. Both are unidirectional switch primitives in this model, and their terminal order is output, input, then control. An instance name is optional.

```
nmos [instance_name] (output, input, control);
pmos [instance_name] (output, input, control);
```

2.2 CMOS transmission-gate switch

A CMOS switch combines an NMOS and PMOS path so that it can pass both logic levels. Its control terminals are listed as `ncontrol` followed by `pcontrol`; the complementary controls turn the two transistor paths on and off together.

```
cmos [instance_name] (output, input, ncontrol, pcontrol);
```

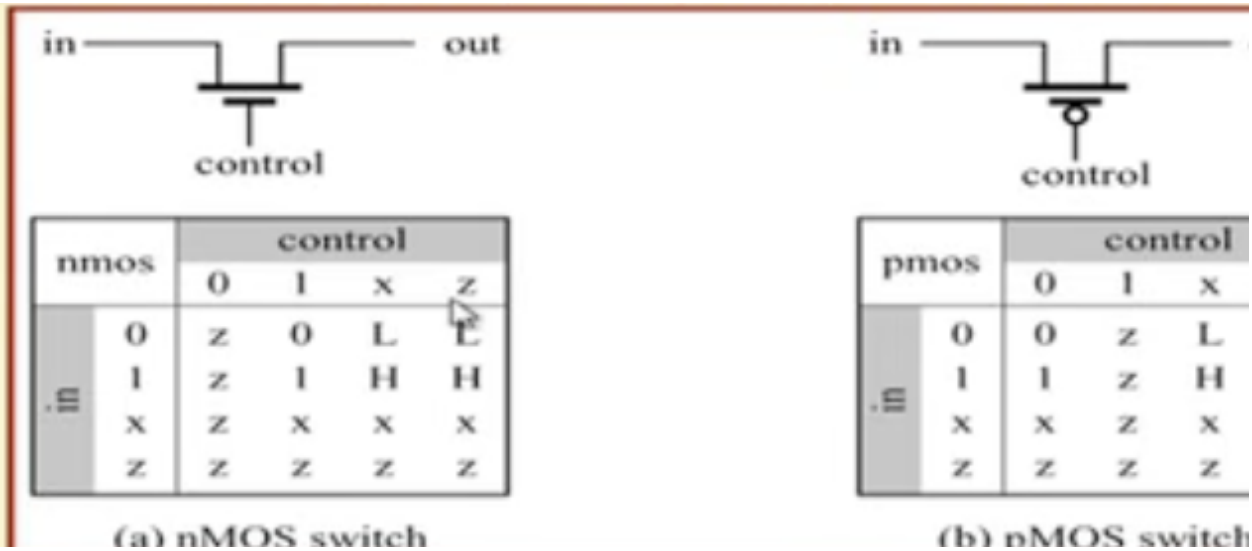


Figure 1: NMOS and PMOS switch symbols and control behavior from the source note.

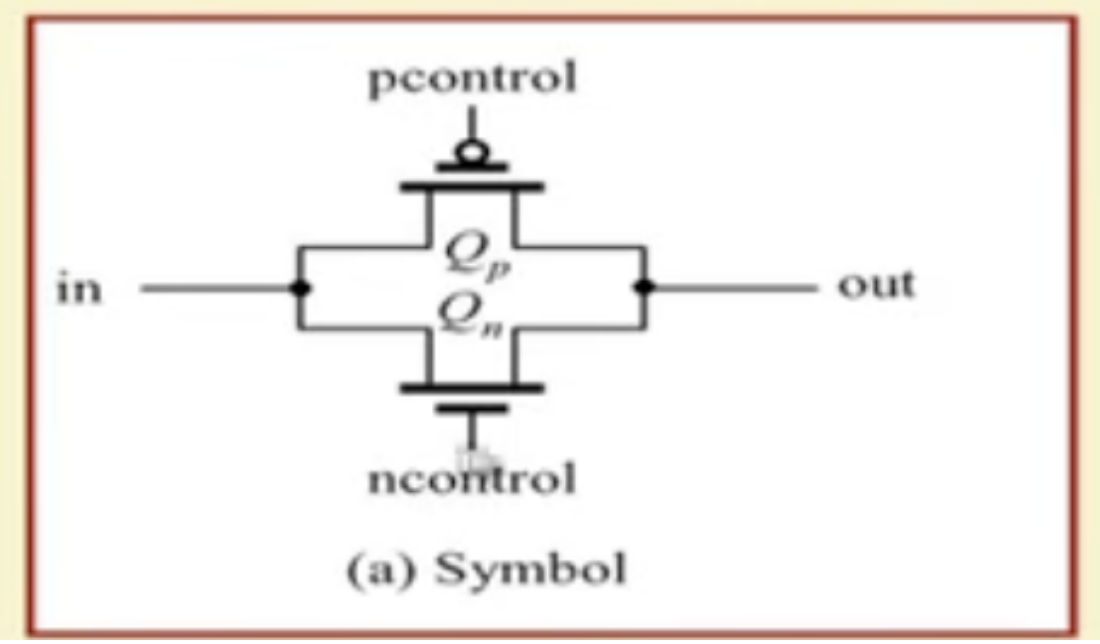


Figure 2: CMOS transmission-gate symbol and primitive syntax from the source note.

3 Gate Examples

3.1 CMOS inverter

The PMOS pulls the output toward `vdd` when the input is low, and the NMOS pulls it toward `gnd` when the input is high. Together they implement $f = \overline{x}$.

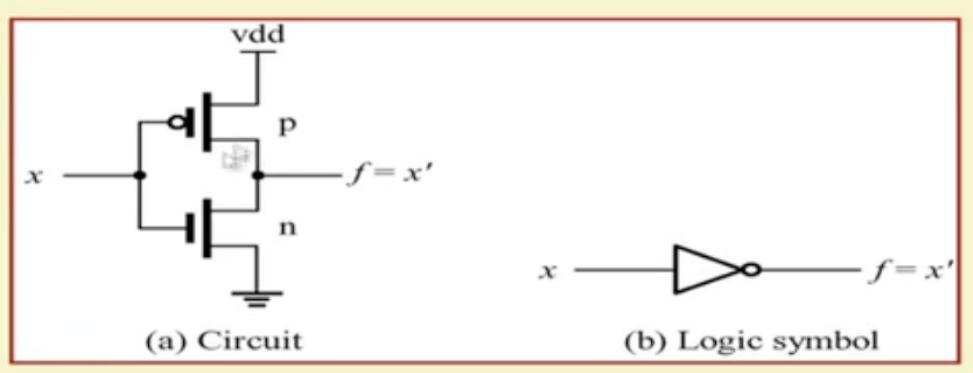


Figure 3: CMOS inverter circuit and logic symbol.

```
module cmosnot (x, f);
  input x;
  output f;
  supply1 vdd;
  supply0 gnd;

  pmos p1 (f, vdd, x);
  nmos n1 (f, gnd, x);
endmodule
```

3.2 CMOS NAND gate

The PMOS devices form a parallel pull-up network and the NMOS devices form a series pull-down path. The output is therefore low only when both inputs are high, giving $f = \overline{xy}$.

```
module cmosnand (x, y, f);
  input x, y;
  output f;
  supply1 vdd;
  supply0 gnd;
  wire a;

  pmos p1 (f, vdd, x);
  pmos p2 (f, vdd, y);
  nmos n1 (f, a, x);
  nmos n2 (a, gnd, y);
endmodule
```

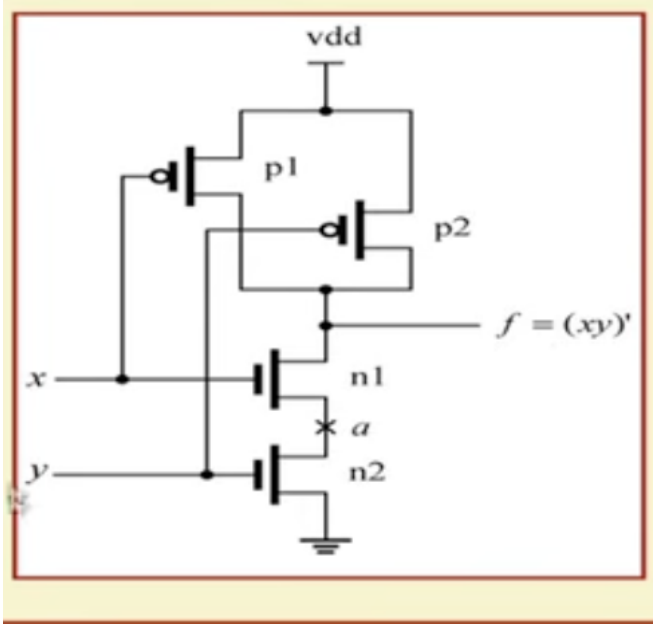


Figure 4: CMOS NAND transistor circuit.

NAND test bench

This test bench applies the four possible two-input combinations, waiting five simulation time units before displaying each result.

```
module cmosnand_test;
  reg in1, in2;
  wire out;
  integer k;

  cmosnand MYNAND2 (in1, in2, out);

  initial begin
    for (k = 0; k < 4; k = k + 1) begin
      #5 {in1, in2} = k;
      $display("In1: %b, In2: %b, Out: %b", in1, in2, out);
    end
  end
endmodule
```

The source note also shows the expected NAND truth-table output:

```
In1: 0, In2: 0, Out: 1
In1: 0, In2: 1, Out: 1
In1: 1, In2: 0, Out: 1
In1: 1, In2: 1, Out: 0
```

3.3 Pseudo-NMOS NOR gate

Here a `pullup` provides the load, while either NMOS can pull the output low. Thus the output is high only when both inputs are low: $f = \overline{x + y}$.

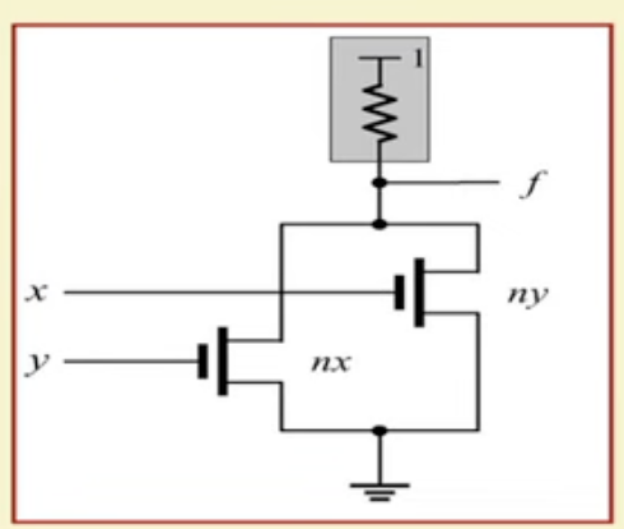


Figure 5: Pseudo-NMOS NOR circuit.

```
module pseudonor (x, y, f);
  input x, y;
  output f;
  supply0 gnd;

  nmos nx (f, gnd, x);
  nmos ny (f, gnd, y);
  pullup (f);
endmodule
```

4 Multiplexers

4.1 CMOS 2-to-1 multiplexer

The two CMOS switches are enabled by opposite select levels. The inverter generates `sbar`, so the output follows `i0` for `s=0` and `i1` for `s=1`.

```
module mux_2to1 (out, s, i0, i1);
  input s, i0, i1;
  output out;
  wire sbar;

  not (sbar, s);
  cmos cmos_a (out, i0, sbar, s);
```

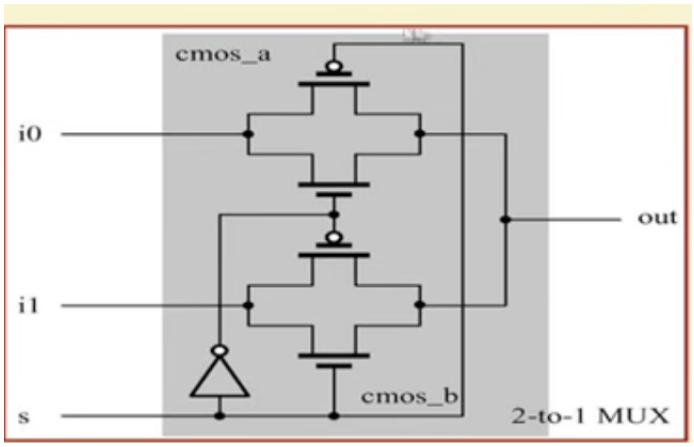


Figure 6: CMOS 2-to-1 multiplexer circuit.

```
  cmos cmos_b (out, i1, s, sbar);
endmodule
```

2-to-1 multiplexer test bench

All eight combinations of the select and data inputs are applied. The source screenshot used a different module name and repeated `in1` in the display arguments. Both are corrected here so the test bench instantiates the preceding module and prints matching input labels.

```
module cmosmux_test;
  reg sel, in0, in1;
  wire out;
  integer k;

  mux_2to1 MUX21 (out, sel, in0, in1);

  initial begin
    for (k = 0; k < 8; k = k + 1) begin
      #5 {sel, in0, in1} = k;
      $display("Sel: %b, In0: %b, In1: %b, Out: %b",
               sel, in0, in1, out);
    end
  end
endmodule
```

The source note output table is retained in terminal-style text:

```
Sel: 0, In0: 0, In1: 0, Out: 0
Sel: 0, In0: 0, In1: 1, Out: 0
Sel: 0, In0: 1, In1: 0, Out: 1
Sel: 0, In0: 1, In1: 1, Out: 1
Sel: 1, In0: 0, In1: 0, Out: 0
Sel: 1, In0: 0, In1: 1, Out: 1
```

```
Sel: 1, In0: 1, In1: 0, Out: 0
Sel: 1, In0: 1, In1: 1, Out: 1
```

4.2 CMOS 4-to-1 multiplexer with `tranif` switches

The first switch in each branch connects one data input to an intermediate wire; a second switch connects that wire to the output. The two select bits choose exactly one branch. The sketch and schematic from the source note are preserved below.

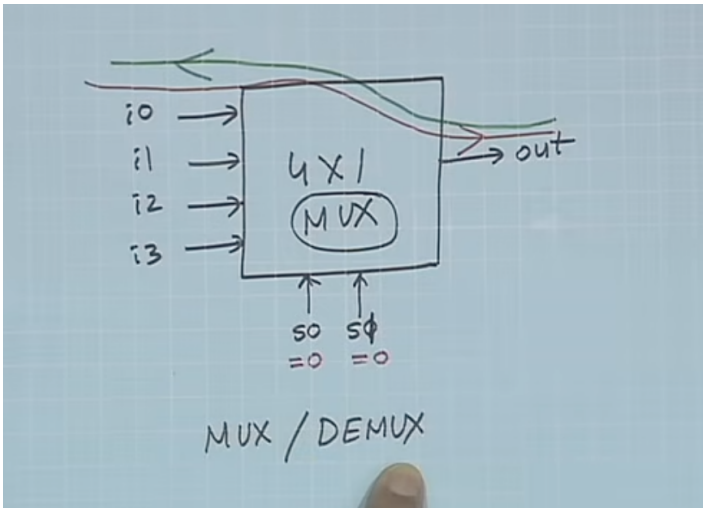


Figure 7: Handwritten 4-to-1 multiplexer sketch retained from the source note.

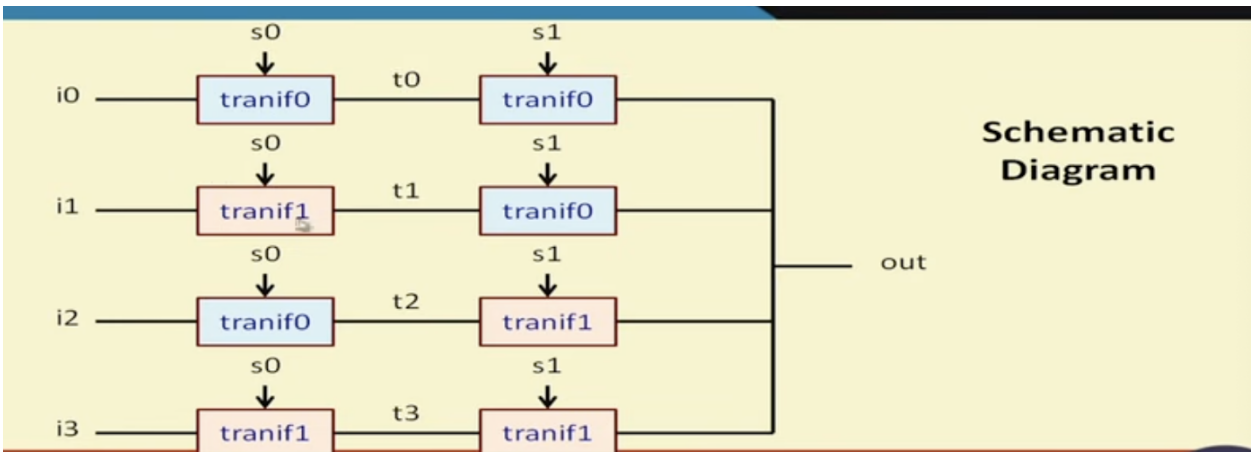


Figure 8: Switch-level schematic for the 4-to-1 multiplexer.

```
module mux_4to1 (out, s0, s1, i0, i1, i2, i3);
  input s0, s1, i0, i1, i2, i3;
  output out;
  wire t0, t1, t2, t3;

  tranif0 (i0, t0, s0);
```

```
  tranif1 (i1, t1, s0);
  tranif0 (i2, t2, s0);
  tranif1 (i3, t3, s0);

  tranif0 (t0, out, s1);
  tranif0 (t1, out, s1);
  tranif1 (t2, out, s1);
  tranif1 (t3, out, s1);
endmodule
```

4-to-1 multiplexer test bench

The test bench iterates over all $2^6 = 64$ combinations of the two select bits and four data inputs.

```
module mux4to1_test;
  reg s0, s1, i0, i1, i2, i3;
  wire out;
  integer k;

  mux_4to1 MYMUX41 (out, s0, s1, i0, i1, i2, i3);

  initial begin
    for (k = 0; k < 64; k = k + 1) begin
      #5 {s0, s1, i0, i1, i2, i3} = k;
      $display("Sel: %2b, In: %4b, Out: %b",
               {s0, s1}, {i0, i1, i2, i3}, out);
    end
  end
endmodule
```

5 Full Adder

A full adder produces a sum and carry from three one-bit inputs:

$$\text{sum} = a \oplus b \oplus c_{in}, \qquad c_{out} = ab + ac_{in} + bc_{in}.$$

The source note's handwritten circuit and sum sketches remain as reference figures. The transistor-level implementation is provided as code.

```
module fulladder (sum, cout, a, b, cin);
  input a, b, cin;
  output sum, cout;

  fa_sum SUM (sum, a, b, cin);
  fa_carry CARRY (cout, a, b, cin);
endmodule
```

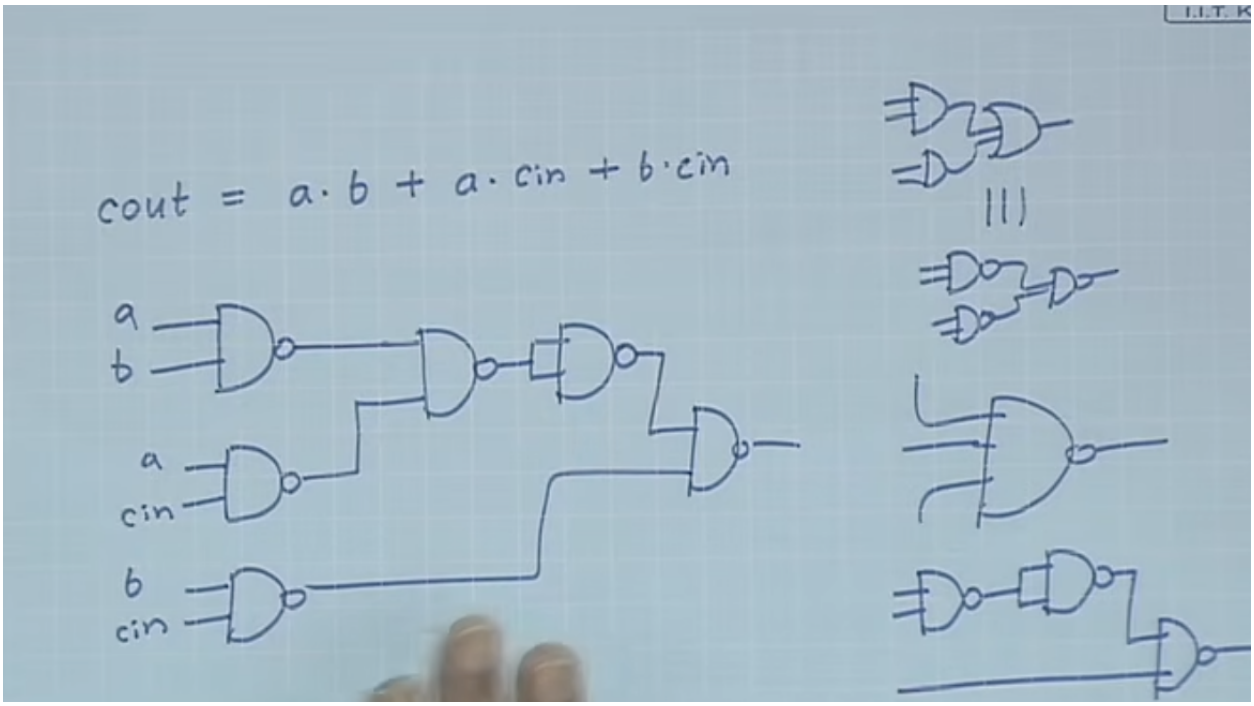


Figure 9: Handwritten carry-logic notes retained from the source PDF.

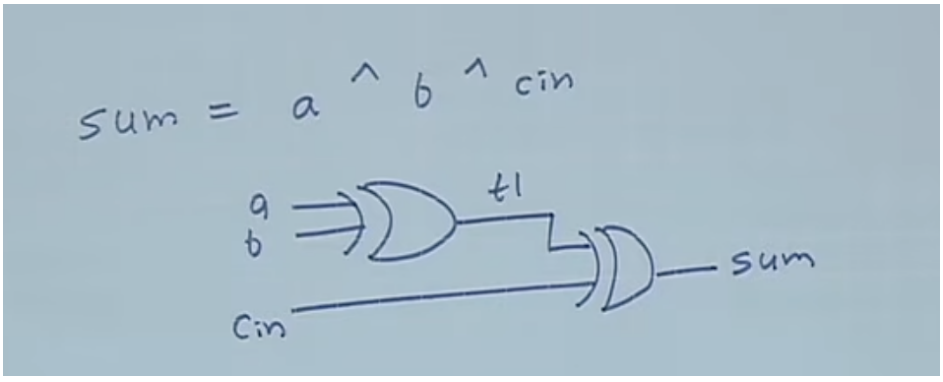


Figure 10: Handwritten XOR-based sum sketch retained from the source PDF.

5.1 Carry network

The carry submodule combines NAND terms to implement the majority function $ab + ac_{in} + bc_{in}$.

```
module fa_carry (cout, a, b, cin);
  input a, b, cin;
  output cout;
  wire t1, t2, t3, t4, t5;

  cmosnand N1 (t1, a, b);
  cmosnand N2 (t2, a, cin);
  cmosnand N3 (t3, b, cin);
  cmosnand N4 (t4, t1, t2);
  cmosnand N5 (t5, t4, t4);
  cmosnand N6 (cout, t5, t3);
endmodule
```

5.2 Sum network

The sum is built from two cascaded XOR operations. The helper `myxor2` is itself implemented using CMOS NAND modules.

```
module myxor2 (out, a, b);
  input a, b;
  output out;
  wire t1, t2, t3, t4;

  cmosnand N1 (t1, a, a);
  cmosnand N2 (t2, b, b);
  cmosnand N3 (t3, a, t2);
  cmosnand N4 (t4, b, t1);
  cmosnand N5 (out, t3, t4);
endmodule

module fa_sum (sum, a, b, cin);
  input a, b, cin;
  output sum;
  wire t1, t2;

  myxor2 X1 (t1, a, b);
  myxor2 X2 (sum, t1, cin);
endmodule
```

5.3 Full-adder test bench

Eight input combinations exercise the full adder.

```
module fulladder_test;
```

```
  reg a, b, cin;
  wire sum, cout;
  integer k;

  fulladder FA (sum, cout, a, b, cin);

  initial begin
    for (k = 0; k < 8; k = k + 1) begin
      #5 {a, b, cin} = k;
      $display("Inputs: %3b Sum: %b, Carry: %b",
               {a, b, cin}, sum, cout);
    end
  end
endmodule
```